



Лабораторная работа № 3

С наступлением весны и теплой погоды вы решили чаще выходить на улицу и поэтому стали иногда ходить делать практические и домашние работы в кофейню (или пиццерию, бургерную или др. заведения). Для того, чтобы не терять время в очереди в ожидании заказа, нужно разработать программу, частично автоматизирующую принятие и оплату заказов.

Техническое задание:

Вам необходимо разработать систему, которая позволит осуществлять заказ блюд из кофейни, после чего пользователь должен получать чек с информацией о сумме оплаты.

Требования к системе:

Пользователь сервиса описывается классом Client. В этом классе должны быть определены свойства: имя (Name), количество средств на счете (Balance), список заказов, а также метод оплаты чека - PayCheck(CoffeeHouse sender, OrderEventArgs e). Информация о чеке передается в параметре OrderEventArgs e. В данном методе от баланса клиента отнимается сумма чека в заказе и, если оплата прошла успешно, заказ сохраняется в списке заказов клиента. Иначе – выводится сообщение об ошибке оплаты].

Класс OrderEventArgs описывает информацию о заказе, передаваемую в событиях, содержит свойство Order order (сам заказ) и string Message (поясняющий текст).

Каждый заказ описывается объектом класса Order. Свойства заказа: список блюд (объекты класса Food с названием блюда и его стоимостью), стоимость заказа, способ получения – доставка или самовывоз. Также создайте метод для вывода информации о заказе.

Так как возможно получение заказа в виде доставки, требуется создать класс курьер (Courier) с атрибутами имя, рейтинг, список принятых заказов, свободен ли курьер. А также описание класса курьера включает делегат void ReadyToDeliveryHandler(Courier sender) и событие DeliveryNotify. Также должен быть метод, позволяющий курьеру сообщить о том, что он может доставить определенный заказ void Delivery(CoffeeHouse sender, OrderEventArgs e). На заказ могут откликнуться те курьеры, которые не заняты и число принятых заказов у которых менее 3. Если курьер подходит под эти требования, возникает событие DeliveryNotify.

Класс компании – кофейни (можете вместо кофейни создать столовую, пиццерию или др.) (CoffeeHouse) содержит свойства:

- список блюд меню Menu. Каждое блюдо: объект класса Food, в котором описано два свойства (название и цена).

- приватную коллекцию курьеров, готовых доставить текущий заказ (изначально эта коллекция пуста). Про курьера мы можем сказать, что у него есть имя, статус (свободен/занят), список текущих заказов.

- делегат delegate void FindCourierHandler(CoffeeHouse sender, OrderEventArgs e) и событие event FindCourierNotify для поиска курьеров;

- делегат void CheckHandler(CoffeeHouse sender, OrderEventArgs e) и событие CheckNotify для отправления чека клиенту.

Методы:

- 1) регистрация курьера CourierRegister (Courier courier) (курьер подписывается на событие FindCourierNotify, которое запускается для поиска курьера; также добавляется подписка кофейни на событие DeliveryNotify класса Курьер, которое запускается, если курьер готов принять заказ).

- 2) создание заказа MakeOrder(Client client, List<Food> food, bool delivery)(параметры: клиент, список продуктов, отметка о необходимости доставки). Здесь создается заказ (Order), клиент подписывается на событие выдача чека CheckNotify, и вызывается метод приготовления заказа OrderPrepare.

3) выполнение заказа OrderPrepare (Client client, Order order):

Здесь считаем сумму заказа. При этом нужно учесть, что в заказе от клиента могут быть ошибки (быть блюда, которых нет в меню), поэтому нужно найти в списке меню все блюда из заказа клиента и получить цены блюд. Кроме того, нужно реализовать то, что не всегда какое-то блюдо есть в наличии (добавьте с помощью рандома небольшую вероятность, что блюдо из меню не будет приготовлено, тогда его цену не учитывайте в чеке). В результате итоговый заказ с уже выполненными блюдами может отличаться от начального.

После этого – если в заказе требуется доставка – происходит вызов события класса CoffeeHouse - event FindCourierNotify, на которое подписаны все зарегистрированные курьеры. Кто из курьеров сможет принять заказ, определяется в функции void Delivery(CoffeeHouse sender, OrderEventArgs e) класса Courier. На заказ могут откликнуться несколько курьеров, все они помещаются в коллекцию OrderCouriers класса кофейни (Company) через метод void FindCourier(Courier sender), подписанный на события курьера. Из всех курьеров, откликнувшихся на заказ, выбирается в итоге тот, у кого больше рейтинг.

Далее, когда курьер найден или если доставка не требуется, происходит вызов события формирования чека CheckNotify, где в сообщении указан актуальный заказ (без учета неприготовленных блюд) и имя курьера, если выбрана доставка. На это событие уже подписан метод клиента PayCheck.

Задача: Пользователь, который решил воспользоваться Вашим приложением, должен ввести выбранные блюда заказа, и способ получения заказа (доставку или самовывоз). После чего формирование заказа происходит путем вызова функции MakeOrder () в классе кофейни. В качестве параметров этой функции передается клиент (client), список блюд в заказе и свойство, указывающие на необходимость доставки. В результате клиент должен получить информацию о выполненном заказе, а также успешно ли прошла его оплата. Протестируйте вашу систему для разных вариантов заказа (например, когда заказанных блюд нет в меню, не хватает денег для оплаты и др.)

!!! методы поиска курьера и оплаты клиентом чека должны быть вызваны при возникновении соответствующих событий, их не нужно вызывать явно)

Пример вывода в консоли:

```
Получен новый заказ: Латте 350,Панкейки, от клиента App
Латте 350 готов!
Панкейки готов!
Поиск курьера!
Курьер: Finn может принять заказ!
3
App, Ваш заказ готов, сумма к оплате: 600 ,Назначен курьер: Finn!
Заказ оплачен!

Получен новый заказ: Латте 200,Панкейки, от клиента Kate
Латте 200 нет в меню!
Панкейки готов!
Kate, Ваш заказ готов, сумма к оплате: 400 Способ получения заказа: самовывоз!
Недостаточно средств, заказ отменен!
```